3 One actor is no actor

This chapter covers

- Actors as a hierarchy
- Creating more actors
- Sending messages back and forth and adapting responses
- Sending messages and waiting for a reply

> *A colony of ants is more than just an aggregate of insects that are living together. One ant is no ant. Two ants, and you begin to get something entirely new. Put a million together with the workers divided into different castes, each doing a different function —cutting the leaves, looking after the queen, taking care of the young, digging the nest out, and so on—and you've got an organism weighing about 11 kilograms [24 pounds], about the size of a dog, and dominating an area the size of a house.*
>
> —David Suzuki: *The Sacred Balance: Rediscovering Our Place in Nature* (Greystone Books, 1997)

Actors are very much the same as ants—they come in organized groups. One actor is no actor; they come in systems. Let's see how.

One of the most compelling features of the actor model is that each actor makes local decisions. At the same time, actors belong to a hierarchy: a top-down relationship between parents and children. Each time an actor creates another actor, it becomes that actor's parent. These two properties—locality and hierarchy—allow the actor model to delegate easily. Traditional Akka applications are based on this delegation.

So far, you have learned how to create actors of the type `ActorSystem`, but this type of actor is unique. It is a singleton.